RAVI KISHAN
CSE(AIML)
202401100400154

DIABETES PREDICTION USING LOGISTIC REGRESSION(**MINOR PROJECT-1**)

Problem Statement

Diabetes is one of the most common chronic diseases worldwide. Early detection can help prevent severe health complications. The objective of this project is to develop a supervised machine learning model that can predict whether a person has diabetes based on various health-related attributes.

Project Objective

The main objective of this project is to build and evaluate a machine learning model that predicts diabetes using patient medical information. Logistic Regression is used as the classification algorithm.

Dataset Description

The dataset used in this project is the **Pima Indians Diabetes Dataset**, which contains medical records of female patients.

Dataset Source

Kaggle Dataset: https://www.kaggle.com/datasets/uciml/pima-indians-diabetes-database

Features

Feature	**Description**
Pregnancies	Number of pregnancies
Glucose	Plasma glucose concentration
BloodPressure	Diastolic blood pressure
SkinThickness	Triceps skin fold thickness

Feature	Description
Insulin	Serum insulin level
BMI	Body Mass Index
DiabetesPedigreeFunction	Diabetes likelihood score
Age	Age of the patient
Outcome	Target variable (0 = No Diabetes, 1 = Diabetes)

Data Preprocessing

The following preprocessing steps were performed:

Step 1: Data Loading

The dataset was loaded using the Pandas library.

Step 2: Missing Value Check

The dataset was checked for missing values and inconsistencies.

Step 3: Feature Selection

The target variable (**Outcome**) was separated from the input features.

Step 4: Feature Scaling

StandardScaler was used to normalize numerical features.

Step 5: Train-Test Split

The dataset was divided into:

- 80% Training Data
- 20% Testing Data

Exploratory Data Analysis (EDA)

EDA was performed to understand the data distribution and relationships among variables.

Analysis Performed

1. Dataset Information
2. Statistical Summary
3. Correlation Analysis
4. Histogram Visualization

5. Outcome Distribution Analysis

Observations

- Higher glucose levels are strongly associated with diabetes.
- BMI and Age also show significant influence on diabetes prediction.
- The dataset contains both diabetic and non-diabetic patients.

Machine Learning Model Used

Logistic Regression

Logistic Regression is a supervised classification algorithm used when the target variable is categorical.

- Suitable for binary classification problems.
- Easy to implement.
- Fast training process.
- Provides interpretable results.

Model Training Process

1. Import required libraries.
2. Load the dataset.
3. Perform preprocessing.
4. Split data into training and testing sets.

5. Train Logistic Regression model.

6. Generate predictions.

7. Evaluate performance using classification metrics.

IMPORT :

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score
from sklearn.metrics import precision_score
from sklearn.metrics import recall_score
from sklearn.metrics import f1_score
from sklearn.metrics import confusion_matrix
```

Load Dataset:

```
import os

for dirname, _, filenames in os.walk('/kaggle/input'):
    for filename in filenames:
        print(os.path.join(dirname, filename))
```

/kaggle/input/datasets/organizations/uciml/pima-indians-diabetes-database/diabetes.csv

```
df = pd.read_csv('/kaggle/input/datasets/organizations/uciml/pima-indians-diabetes-database/diabetes.csv')
```

INFO:

[12]:

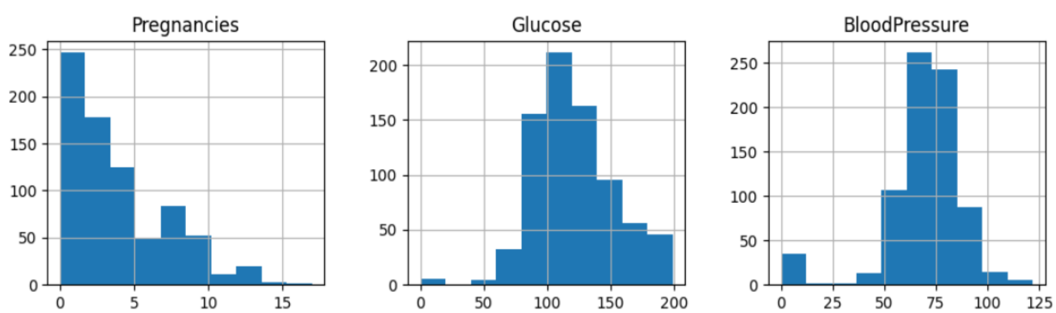
```python
print("First 5 Rows")
print(df.head())

print("\nDataset Shape")
print(df.shape)

print("\nDataset Info")
print(df.info())

print("\nMissing Values")
print(df.isnull().sum())
```

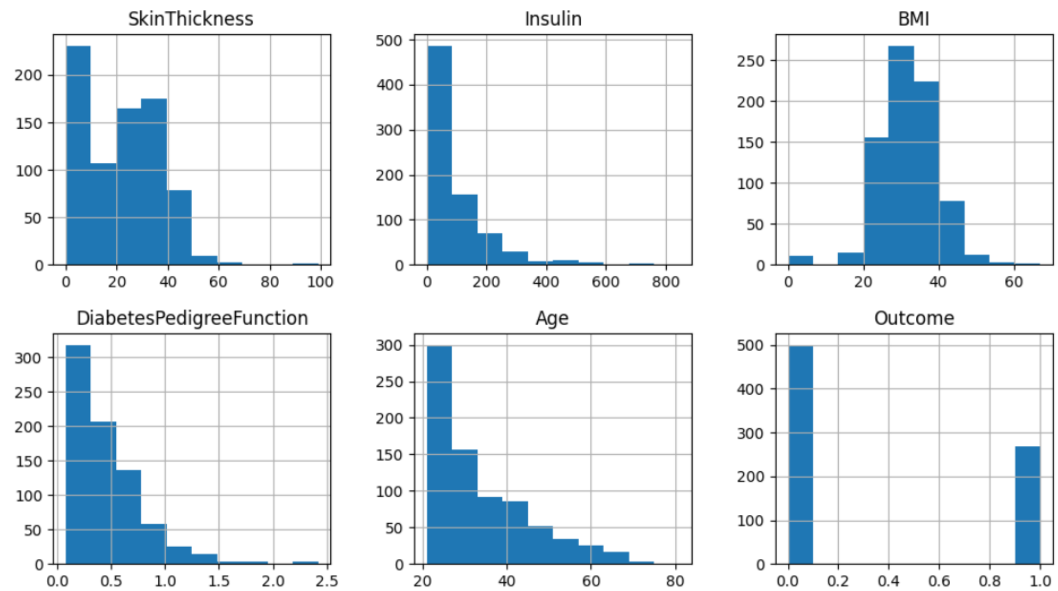
```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1

Dataset Shape
(768, 9)

Dataset Info
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 768 entries, 0 to 767
Data columns (total 9 columns):
 #   Column                    Non-Null Count  Dtype  
---  ------                    --------------  -----  
 0   Pregnancies               768 non-null    int64  
 1   Glucose                   768 non-null    int64  
 2   BloodPressure             768 non-null    int64  
 3   SkinThickness             768 non-null    int64  
 4   Insulin                   768 non-null    int64  
 5   BMI                       768 non-null    float64
 6   DiabetesPedigreeFunction  768 non-null    float64
 7   Age                       768 non-null    int64  
 8   Outcome                   768 non-null    int64  
dtypes: float64(2), int64(7)
memory usage: 54.1 KB
None
```

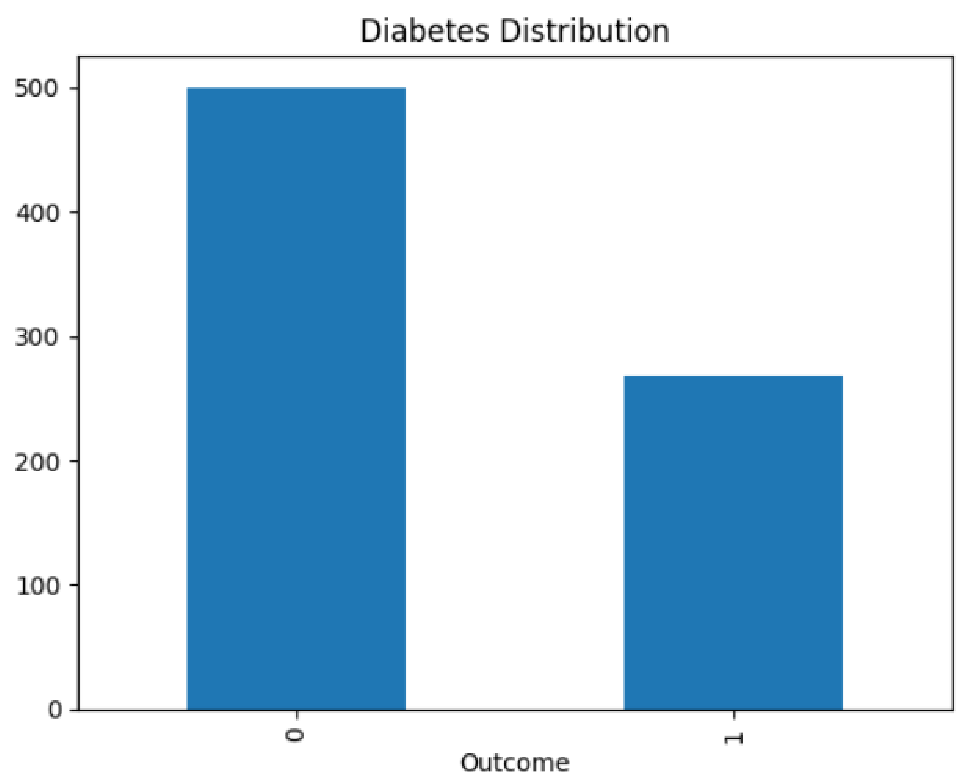
EDA (Exploratory Data Analysis):

```python
print("\nStatistical Summary")
print(df.describe())

# Histogram
df.hist(figsize=(12,10))
plt.show()

# Outcome Distribution
df['Outcome'].value_counts().plot(kind='bar')
plt.title("Diabetes Distribution")
plt.show()
```

```
Statistical Summary
       Pregnancies     Glucose  BloodPressure  SkinThickness     Insulin  \
count   768.000000  768.000000     768.000000     768.000000  768.000000
mean      3.845052  120.894531      69.105469      20.536458   79.799479
std       3.369578   31.972618      19.355807      15.952218  115.244002
min       0.000000    0.000000       0.000000       0.000000    0.000000
25%       1.000000   99.000000      62.000000       0.000000    0.000000
50%       3.000000  117.000000      72.000000      23.000000   30.500000
75%       6.000000  140.250000      80.000000      32.000000  127.250000
max      17.000000  199.000000     122.000000      99.000000  846.000000

              BMI  DiabetesPedigreeFunction         Age     Outcome
count  768.000000                768.000000  768.000000  768.000000
mean    31.992578                  0.471876   33.240885    0.348958
std      7.884160                  0.331329   11.760232    0.476951
min      0.000000                  0.078000   21.000000    0.000000
25%     27.300000                  0.243750   24.000000    0.000000
50%     32.000000                  0.372500   29.000000    0.000000
75%     36.600000                  0.626250   41.000000    1.000000
max     67.100000                  2.420000   81.000000    1.000000
```







\+ Code + Markdown

features aur target:

+ Code + Markdown

[14]:

```
X = df.drop('Outcome', axis=1)
y = df['Outcome']
```

Data Scaling and Train-Test Split:

[1]:

```
scaler = StandardScaler()
X = scaler.fit_transform(X)



X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)
```

Model:

[17]:
```
model = LogisticRegression()

model.fit(X_train, y_train)
```

[17]:
```
▾ LogisticRegression
LogisticRegression()
```

[18]:
```
y_pred = model.predict(X_test)
```

performance evaluate:

```python
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)

print("\nModel Performance")
print("Accuracy :", accuracy)
print("Precision:", precision)
print("Recall   :", recall)
print("F1 Score :", f1)
```

```
Model Performance
Accuracy : 0.7532467532467533
Precision: 0.6491228070175439
Recall   : 0.6727272727272727
F1 Score : 0.6607142857142857
```

+ Code + Markdown

Confusion matrix:

```python
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix")
print(cm)
```

```
Confusion Matrix
[[79 20]
 [18 37]]
```

Result:

```
sample = X_test[0].reshape(1,-1)

prediction = model.predict(sample)

if prediction[0] == 1:
    print("\nPatient is Diabetic")
else:
    print("\nPatient is Not Diabetic")
```

```
Patient is Not Diabetic
```

Code link: https://www.kaggle.com/code/ravikishan12/diabetes-detection

Conclusion:

This project successfully developed a machine learning model for diabetes prediction using Logistic Regression. The model achieved satisfactory accuracy and demonstrated the effectiveness of supervised learning in healthcare applications. Such predictive systems can assist medical professionals in early diagnosis and decision-making.

Future Scope

- Use larger healthcare datasets.
- Apply advanced algorithms such as Random Forest and XGBoost.

- Deploy the model as a web application.
- Improve prediction accuracy through hyperparameter tuning.

References

1. Pima Indians Diabetes Dataset (Kaggle)
2. Scikit-Learn Documentation
3. Pandas Documentation
4. Machine Learning with Python Resources